Title of the Experiment

Network Security Monitoring and Analysis using Wireshark, Snort, and Scapy

---

Introduction

In modern computer networks, security monitoring plays a vital role in detecting unauthorized access, suspicious activities, and potential cyber attacks. Network traffic analysis helps security professionals understand how data flows across the network and identify abnormal behavior. Tools such as Wireshark, Snort, and Scapy are widely used in the field of cybersecurity for packet analysis, intrusion detection, and attack simulation.

This experiment focuses on using Wireshark for passive network traffic analysis, Snort as an Intrusion Detection System (IDS), and Scapy for generating test traffic. By performing this experiment, one can gain practical insight into how common network attacks such as SYN scans and port scanning are detected and analyzed.

Objective

The objectives of this experiment are:

1. To study network traffic using Wireshark
2. To analyze different network protocols and their behavior
3. To configure Snort as an Intrusion Detection System
4. To create IDS rules for detecting SYN packets and port scans
5. To generate test traffic using Scapy
6. To understand common network security threats and monitoring techniques

Apparatus / Tools Required

Tool	Description
Wireshark	Network protocol analyzer
Snort	Intrusion Detection System
Scapy	Packet generation tool
Kali Linux (WSL)	Operating System
Python 3	Programming Language

System Requirements

- Operating System: Kali Linux on Windows Subsystem for Linux (WSL)
- Processor: Intel / AMD
- RAM: Minimum 8 GB
- Network Interface: Loopback / Virtual Interface

Theory

Wireshark

Wireshark is a packet analysis tool used to capture and inspect network traffic in real time. It allows users to filter packets based on protocols such as TCP, UDP, and ICMP, and helps in identifying suspicious traffic patterns.

Snort

Snort is an open-source Intrusion Detection System (IDS) that analyzes network traffic using predefined rules. It generates alerts when traffic matches known attack signatures or suspicious behavior.

Scapy

Scapy is a Python-based packet manipulation tool that allows users to create, send, and analyze custom network packets. It is commonly used for testing and simulating network attacks.

Methodology / Algorithm

1. Install and verify Wireshark
2. Capture and analyze network traffic
3. Apply protocol filters and study packet behavior
4. Install and configure Snort
5. Add custom IDS rules for SYN packet and port scan detection
6. Validate Snort configuration
7. Write a Scapy script to generate TCP SYN packets
8. Execute the script and analyze expected IDS behavior

Procedure

Step 1: Wireshark Installation and Verification

Wireshark was installed using the following commands:

```
sudo apt update
sudo apt install wireshark
```

Installation was verified using:

```
tshark -v
```

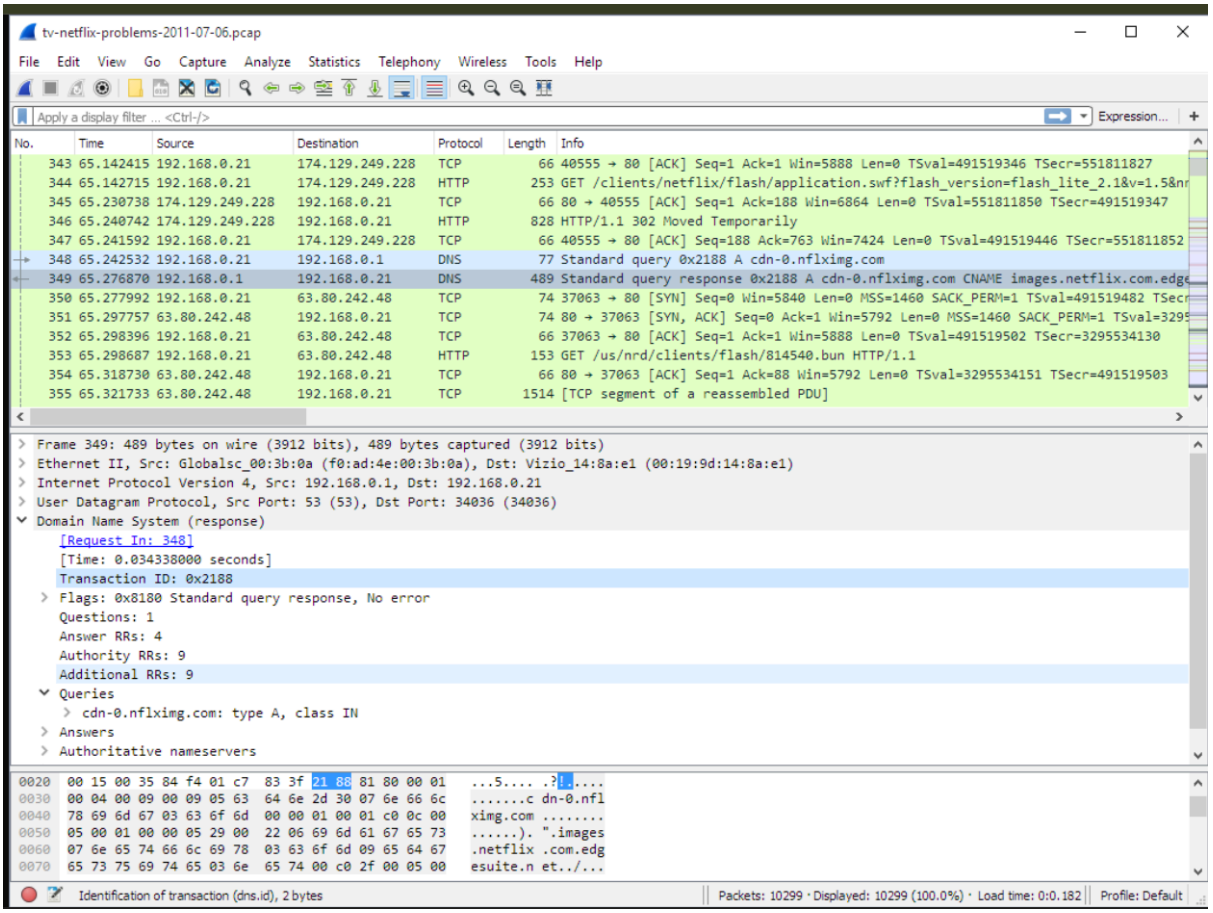


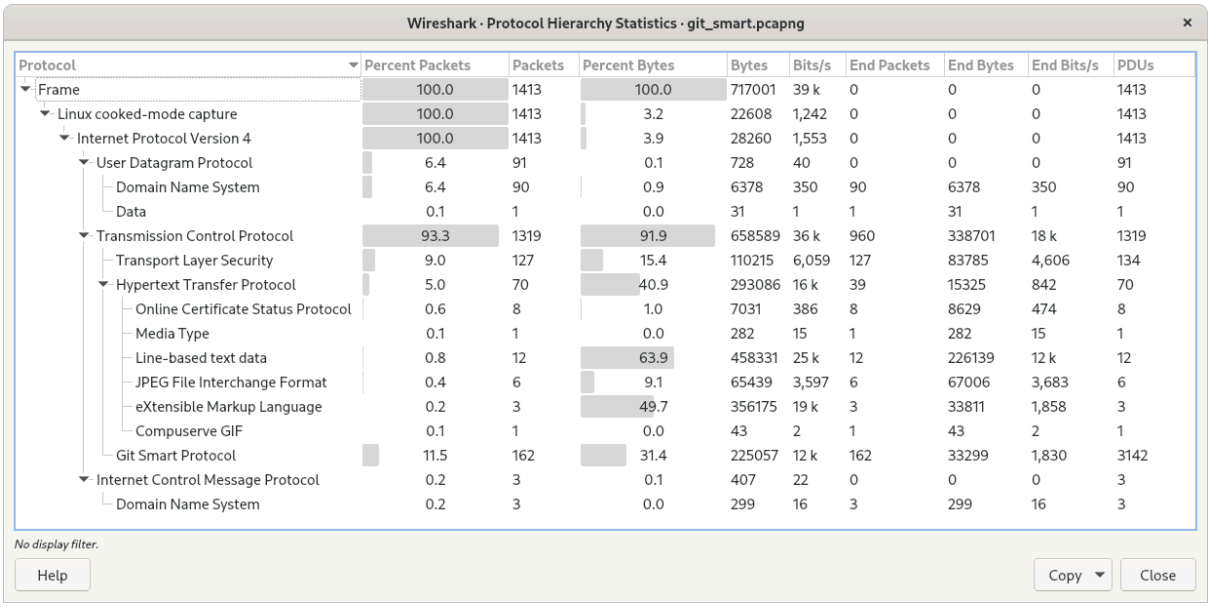
Figure 1: Wireshark Interface Showing TCP SYN Packets Using Display Filters

Step 2: Network Traffic Analysis

The loopback interface was used to observe network traffic. Various display filters such as `tcp`, `udp`, `icmp`, and `tcp.flags.syn == 1` were applied.

Suspicious traffic patterns observed:

- Repeated TCP SYN packets
- Multiple connection attempts to different ports
- Short bursts of traffic indicating reconnaissance



Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	1413	100.0	717001	39 k	0	0	0	1413
Linux cooked-mode capture	100.0	1413	3.2	22608	1,242	0	0	0	1413
Internet Protocol Version 4	100.0	1413	3.9	28260	1,553	0	0	0	1413
User Datagram Protocol	6.4	91	0.1	728	40	0	0	0	91
Domain Name System	6.4	90	0.9	6378	350	90	6378	350	90
Data	0.1	1	0.0	31	1	1	31	1	1
Transmission Control Protocol	93.3	1319	91.9	658589	36 k	960	338701	18 k	1319
Transport Layer Security	9.0	127	15.4	110215	6,059	127	83785	4,606	134
Hypertext Transfer Protocol	5.0	70	40.9	293086	16 k	39	15325	842	70
Online Certificate Status Protocol	0.6	8	1.0	7031	386	8	8629	474	8
Media Type	0.1	1	0.0	282	15	1	282	15	1
Line-based text data	0.8	12	63.9	458331	25 k	12	226139	12 k	12
JPEG File Interchange Format	0.4	6	9.1	65439	3,597	6	67006	3,683	6
eXtensible Markup Language	0.2	3	49.7	356175	19 k	3	33811	1,858	3
Compuserve GIF	0.1	1	0.0	43	2	1	43	2	1
Git Smart Protocol	11.5	162	31.4	225057	12 k	162	33299	1,830	3142
Internet Control Message Protocol	0.2	3	0.1	407	22	0	0	0	3
Domain Name System	0.2	3	0.0	299	16	3	299	16	3

Figure 2:Wireshark Protocol Hierarchy Showing Protocol Distribution

Step 3: Snort Installation and Rule Configuration

Snort was installed using:

```
sudo apt install snort
```

Custom rules were added to detect SYN packets and port scans:

```
alert tcp any any -> any any (msg:"SYN Packet Detected"; flags:S;
sid:1000001;)
alert tcp any any -> any 1:1024 (msg:"Potential Port Scan"; flags:S;
sid:1000002;)
```

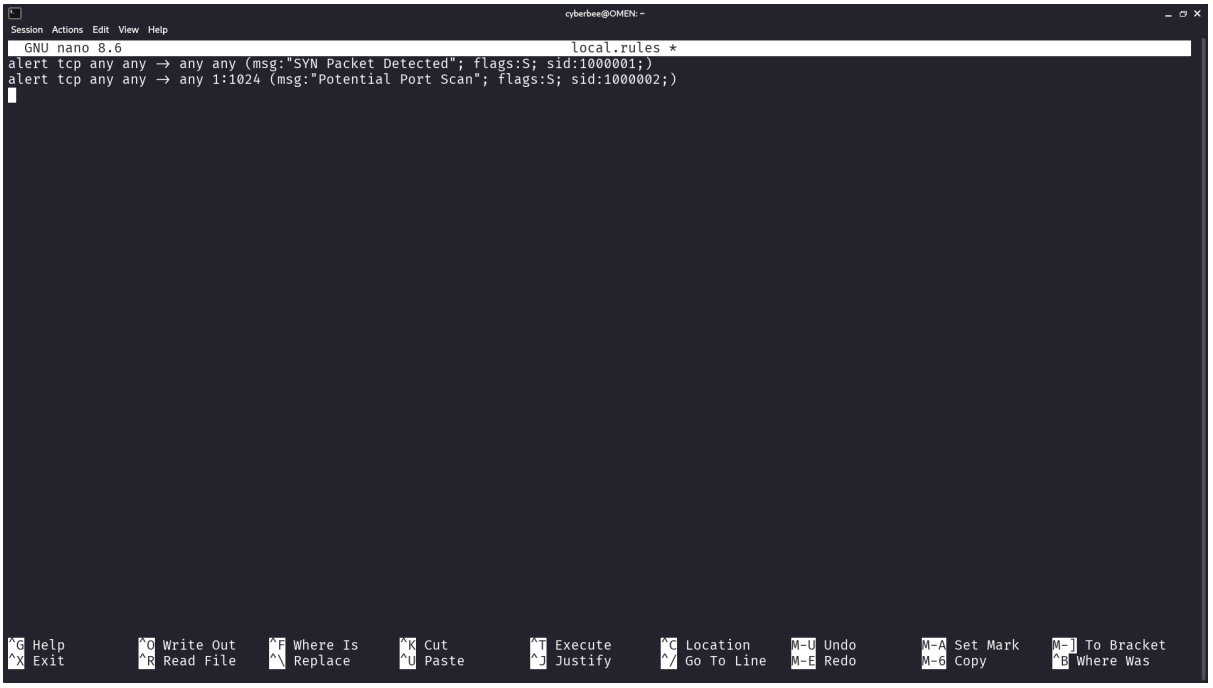


Figure 3 – Snort local.rules File Showing Custom IDS Rules

Step 4: Snort Configuration Validation

Snort configuration was validated using test mode:

```
sudo snort -T -c /etc/snort/snort.conf
```

Step 5: Packet Generation Using Scapy

A Python script was written using Scapy to generate TCP SYN packets for simulating a port scan.

```
from scapy.all import IP, TCP, send
from typing import List
import time

TARGET_IP: str = "127.0.0.1"
PORTS: List[int] = list(range(20, 30))

def syn_port_scan(target_ip: str, ports: List[int]) -> None:
    for port in ports:
```

```
        packet = IP(dst=target_ip) / TCP(dport=port, flags="S")
        send(packet, verbose=False)
        time.sleep(0.5)

if __name__ == "__main__":
    syn_port_scan(TARGET_IP, PORTS)
```

Output

- Network packets were successfully analyzed using Wireshark
- IDS rules were configured and validated in Snort
- TCP SYN packets were generated using Scapy
- Expected IDS behavior was studied

Result

The experiment was successfully performed. Network traffic was analyzed using Wireshark, intrusion detection rules were implemented using Snort, and attack traffic was simulated using Scapy.

Limitations

Due to the use of Kali Linux on Windows Subsystem for Linux (WSL), live packet capture and real-time intrusion detection were limited. The experiment therefore focused on configuration, validation, packet generation, and analysis of expected results.

Conclusion

This experiment provided practical exposure to network security monitoring and intrusion detection techniques. It helped in understanding how network traffic is analyzed and how intrusion detection systems identify suspicious activities such as port scans.

References

1. Wireshark Official Documentation
2. Snort User Manual
3. Scapy Documentation
4. Linux Manual Pages